Hybrid Deep Learning for Stock Market Prediction: A Comparative Study of Modern Forecasting Architectures

Mohamed Mohieldin Mohamed Elkemany

University of East London

Masters of Data Science

March 2026

- ## Dedication

To my wife — thank you for your patience, your support, and for keeping me sane through all of this. You believed in me in the days I didn't even believe in myself, and that made all the difference.

This work is as much yours as it is mine.

- ## Acknowledgment

Use of AI

I acknowledge the use of Microsoft Copilot (<https://copilot.microsoft.com/>) to assist with proofreading, improving clarity, and refining the wording of some sections of my dissertation.

This support was used intermittently between January 2026 and March 2026. The tool was used only for permitted purposes, including checking grammar, improving clarity, and helping me explore ideas. It was not used to generate or paraphrase academic content.

Examples of prompts used include:

- "How clear is this paragraph"
- "Check for grammar mistakes"
- "What areas of comparisons in stock market prediction other than closing price"

- Table of Contents

- **CHAPTER 1 — INTRODUCTION**

1.1 Background of the Study 6

1.2 Statement of the Problem 7

1.3 Purpose of the Study 7

1.4 Research Aim 7

1.5 Theoretical Framework 7

1.6 Importance of the Study 8

1.7 Scope and Limitations 8

1.8 Structure of the Dissertation 8

- **CHAPTER 2 — LITERATURE REVIEW**

2.1 Financial Time Series & Market Direction Prediction 9

2.2 ARIMA and Statistical Models 9

2.3 Deep Learning for Time Series 9

2.4 CNNs for Feature Extraction 10

2.5 LSTMs for Sequential Modeling 10

2.6 Hybrid CNN–LSTM Architectures 10

2.7 Walk Forward Validation 11

2.8 Strengths & Weaknesses of Existing Studies 11

- **CHAPTER 3 — METHODOLOGY**

3.1 Research Design 12

3.2 Data Source and Description 12

3.3 Data Preprocessing 12

3.4 Feature Engineering 13

3.5 Model Architectures 14

3.5.1 ARIMA 14

3.5.2 LSTM 14

3.5.3 CNN–LSTM 15

3.6 Training Procedures 15

3.7 Walk Forward Validation 16

3.8 Evaluation Metrics 17

3.8.1 Forecasting Metrics 17

3.8.2 Classification Metrics 18

3.9 Data Analysis Plan 19

3.10 Ethical Considerations 20

- **CHAPTER 4 — FINDINGS**

4.1 Descriptive Statistics 21

4.2 Regression Results (ARIMA, LSTM, CNN–LSTM) 22

4.3 Classification Results (LSTM vs CNN–LSTM) 23

4.4 Walk Forward Validation Results 29

4.5 Key Findings 33

- **CHAPTER 5 — DISCUSSION AND CONCLUSION**

5.1 Discussion of Results 33

5.2 Why Classification Outperformed Regression 33

5.3 Why CNN–LSTM Outperformed LSTM 34

5.4 Implications for Practice 35

5.5 Limitations 35

5.6 Recommendations for Future Research 35

5.7 Conclusion 36

REFRENCES 37

Code Availability 39

- ## List of Tables

1. **Table 1.**
   Statistical overview of the dataset employed in this research.
2. **Table 4.2**
   Regression Performance Metrics for ARIMA, LSTM, and CNN-LSTM Models.
3. **Table 4.3**
   Classification Performance Metrics for LSTM and CNN LSTM.

- ## List of Figures

1. **Figure 3.1** *Internal structure of an LSTM unit showing the input, forget, and output gates.*
2. **Figure 3.2** *Architecture of the CNN-LSTM hybrid model combining convolutional feature extraction with LSTM-based temporal modelling.*
3. **Figure 4.1** *Predicted vs Actual S&P 500 Closing Prices on the Test Set*
4. **Figure 4.2** *LSTM Confusion Matrix*
5. **Figure 4.3** *CNN LSTM Confusion Matrix*
6. **Figure 4.4** *LSTM Training and Validation Curves*
7. **Figure 4.5** *CNN LSTM Training and Validation Curves*
8. **Figure 4.6** *Bar Chart (Accuracy, Specificity, Sensitivity, F1 Score)*
9. **Figure 4.7** *Bar Chart (Accuracy, Precision, Recall, Specificity, F1 Score, MCC)*
10. **Figure 4.8** *ROC Curve Comparison (LSTM vs CNN LSTM)*
11. **Figure 4.9** *Conceptual Diagram of Walk Forward Validation*
12. **Figure 4.10** *Custom Walk-Forward Validation Setup for S&P 500 Forecasting*
13. **Figure 4.11** *Walk-forward validation with four folds showing ignored lookback.*

CHAPTER 1 — INTRODUCTION

1.1 Background of the Study

Financial markets evolved dramatically through the past decades; complexity increased and became much more data oriented. The number of factors that influence the markets have increased as well, a vast majority of research students try to break this wall but find the market reaction to macroeconomics, politics and behavioral trends challenging. We don't want to assume that volatility didn't exist in the earlier decades of the century, but what we know for sure is that the volume of trading went up through the roof. Risk management became harder; spotting investment opportunities became tougher with such a market that is prone to fluctuations and sudden changes. Although the availability of traditional statistical models like GARCH and ARIMA, they find difficulties in predicting or understanding such nonlinear market patterns, changing dynamics due to the nature of the data. This is a time series forecasting but it is not similar to weather forecasting, or medical pandemic forecasting.

In recent years, machine learning and deep learning techniques have introduced new set of skills in the field of financial time series forecasting. Models like Recurrent Neural Network (RNN) and Long Short Term Memory (LSTM) and Convolutional Neural Networks (CNN) proved their ability to explore complex time-series in sequential data. They are now considered suitable for this type of data predictive analysis as the short-term temporal patterns intervene with the long-term dependencies and with the weight of sudden fluctuations and directional spikes in the equation.

And although all these advanced models and techniques, predicting an index like the S&P 500 remains a tough challenge due to all of those outside effects and features. And so the researchers started to study a deep learning hybrid approach in solving such problems, like the example this dissertation is going to use, which is combining CNN with LSTM in one predictive model and at the same time numerical predictions hold a value, directional predictions emerged to take a place at the conversation due to its realistic practical value in real life decision making contexts.

This research builds on those developments through a study of deep learning models' performance in predicting the S&P 500 index using a regression task to predict the closing price value and a classification task using walk-forward expanding window validation technique. This research aims to find a better understanding of evaluating deep learning models in a simulation of real-life market environments, and whether using a hybrid model will benefit the cause more than just using a standalone model and sparing unnecessary technicalities.

1.2 Statement of the Problem

Even though technologies have advanced a lot in deep learning techniques, predicting index movement is still quite a challenge due to fluctuating nature of finance and market nonlinearity. Traditional models usually suffer in picking up dynamic complex patterns, while deep learning models' efficiency is tied to its architecture and the type of task. Moreover, there is no clear vision of how powerful different models like LSTM and CNN-LSTM can be. The gap between the real-life scenarios and the computational simulation still exists because of the lack of external effects data and need of frequent update to all those other factors needed to put the study in a practical context. A systematic investigation is needed to compare between different deep learning models and understand whether they provide an edge in numerical predictions or directional predictions or both.

1.3 Purpose of the Study

This research aspires to evaluate the effectiveness of deep learning models in predicting the S&P 500 index, while focusing on comparing the performance of LSTM standalone and CNN-LSTM hybrid model. This aspiration came from the need to know if integrating CNN layers used to extract short term patterns really gives an additional value and outperforms the standalone LSTM in dealing with financial time series. By using Walk-forward expanding windows and analyzing each of numerical predictions in the form of closing price values and directional predictions in the form of bull and bear regimes. The study seeks to know If the hybrid model really worth the relative computational efforts or not.

1.4 Research Aim

The main goal of this research is to evaluate and assess the deep learning models either standalone or hybrid and try to push them as far as they can go in terms of robustness and accuracy thus increasing the reliability a trader has in deep learning techniques. We want to do this in the cleanest way so if having a hybrid model doesn't make a difference we want to know.

1.5 Research Questions

We seek also to answer a set of questions that will help draw a big picture of how effective deep learning models are in predicting the S&P 500 index:

1. The performance of standalone LSTM in predictions in either closing price prediction or regime prediction.
2. The ability of the hybrid model CNN-LSTM to deliver better results and performance than standalone LSTM
3. The difference made by the additional layer of CNN when dealing with fluctuating data
4. The influence of walk-forward expanding window has on the results
5. The stability of each model in testing periods folds simulating real-life scenarios.

1.6 Importance of the Study

This study earns its importance from the increasing demand for deep learning models in financial time series forecasting, especially when dealing with highly volatile data. The importance is highlighted when the study doesn't stop at evaluating only one model but compares multiple models and puts them into real market conditions to decide which is better, where and how. Using the walk forward validation technique also adds to its importance as it serves the goal of creating real practical scenarios that reflect the market environment well and narrow the gap. The results will help traders and investors in decision-making and risk management.

1.7 Scope and Limitations

This study focuses on predicting S&P 500 index using daily historical data through deep learning models like LSTM and CNN-LSTM. It measures numerical prediction and directional prediction performance, additionally it implements walk-forward expanding window technique which helps with simulation.

Limits of the study is using only one index which affects the generalization of the study, using only two deep learning models, fixed window size and time frame, and no adding to news feed or external factors.

1.8 Structure of the Dissertation

This dissertation is organized into five main chapters, each addressing a specific component of the research:

Chapter One: Introduction

This chapter presents the background of the study, the research problem, objectives, questions, significance, scope and limitations, and an overview of the dissertation structure.

Chapter Two: Literature Review

This chapter discusses the theoretical foundations related to deep learning models used in time-series forecasting. It also reviews previous studies relevant to financial prediction, highlighting the research gap that this study aims to address.

Chapter Three: Methodology

This chapter describes the dataset, preprocessing steps, model architectures, evaluation metrics, and the walk-forward expanding window validation approach used to assess model performance.

Chapter Four: Findings

This chapter presents the experimental results, compares the performance of the LSTM and CNN–LSTM models, and discusses the implications of the findings.

Chapter Five: Discussion and Conclusion

This chapter summarizes the dissertation, outlines the limitations of the study, and provides recommendations for future research.

CHAPTER 2 — LITERATURE REVIEW

2.1 Financial Time Series & Market Direction Prediction

Financial time series data is stored in a historical sequential manner like closing price, trading volume, and index values (Tsay, 2010). This data is known for its nonlinearity and fluctuations as well as it's affected by external factors politically, economically and even psychologically, which makes the prediction task complicated (Fama, 1970; Cont, 2001).

Directional prediction focuses on determining whether the market will move up or down which makes it a classification task (Kim, 2003). This piece of information is valuable to traders as it helps them in decision making (Henrique et al., 2019). Traditional theories assume random movements of an index, but new studies showed that temporal patterns exist and can be picked up and used by deep learning models which improve prediction quality (Fischer & Krauss, 2018; Nelson et al., 2017).

2.2 ARIMA and Statistical Models

Financial time series forecasting is considered one of the traditional statistical tools for time series forecasting. ARIMA assumes stability and linearity in the series. It uses autoregressive, integrated, and moving-average components to pick up on short term and medium patterns (Box, Jenkins, & Reinsel, 2015).

Traditional statistical models are characterized by the clarity of explanations and ease of application when their assumptions are true, as well as needing the data to be not so big in volume and gives reliable answers in cases where there is no or low noise. And so, it is limited when it comes to nonlinear data and complex data. It fails to deal with multivariate patterns or volatility found in financial markets data.

ARIMA is usually used as a basic reference in comparison when evaluating machine and deep learning models to showcase the difference between them in terms of explanation powers and prediction abilities to nonlinear complex conditions of financial markets (Zhang, 2003).

2.3 Deep Learning for Time Series

Deep Learning technology is one of the latest methods used in financial time-series forecasting, due to its abilities to pick up on nonlinear relations and complex patterns that traditional models fail at (Fischer & Krauss, 2018). It depends on using multiple layers of processing, which allows it to extract deep features from temporal data without the need for manual feature engineering or by using the bare minimum.

Deep learning models are distinguished by their ability to deal with high volatility, noise, long-term dependencies which are very occurring in financial markets. DL models also proved they can improve the quality of numerical and directional predictions especially when given large

volumes of historical data. Many models are used in this field, like CNN, RNN, LSTM and GRU. In addition to hybrid models which combine two or more of these technologies to handle all variants of temporal patterns.

2.4 CNN for Feature Extraction

CNN is used in time-series forecasting to extract the significant features from the raw data in an automated way. Even though it was developed primarily to process photos, it showed effectiveness when handling time-series through its ability to pick up local temporal short-term data (LeCun et al., 2015).

CNN depends on applying filters that move through data to explore changes and directions that might not be clear to traditional models. This makes it a good fit to extract features like fluctuations, peaks, reflections or any recurrent behavior inside the time series. It is also distinguished by how simple it is calculated and can be easily combined with other models like LSTM to produce hybrid models able to extract local and temporal features (Zhang, 2003).

2.5 LSTM for Sequential Modeling

LSTM is considered one of the most important models used in financial time-series forecasting because it was developed especially to deal with long-term dependencies inside the data (Hochreiter & Schmidhuber, 1997). It relies on internal memory units structured to store significant information and ignore insignificant information which makes it a very good fit to pick up directions and extended fluctuations in financial markets.

Studies have shown that LSTM outperforms models like RNN due to its ability to avoid problems like vanishing or exploding gradients which allows it to improve the accuracy of predictions and gives it more stability during training on noisy data (Fischer & Krauss, 2018).

It is used widely along with CNN in hybrid models to extract local features first then model it temporally.

2.6 Hybrid CNN–LSTM Architectures

The hybrid architecture combines CNN and LSTM to get the best out of each one, where CNN excels at extracting local short-term features from the time-series, and the LSTM will be in charge of modeling long term dependencies inside the data (Pandey et al., 2024). This fusion let the model deal with various kinds of patterns and not just be limited to a set of patterns.

Hybrid models usually outperform standalone models, because of its ability to pick up detailed patterns and deep time series, which turns their accuracy and robustness rates up. As well as it is a good fit for financial data where a lot of noise tags along and many patterns coexist. It is usually the go to approach when it comes to such data.

2.7 Walk-Forward Validation

Walk-Forward Validation, one of the most common validation techniques of time series models' performance. It is a technique where data is split into a training window and a rolling testing window that moves gradually with time. The model is trained on the first part of the data, then tested on the following part straight away, and then the window moves one step forward and training is redone while testing on new parts of the series (López de Prado, 2018).

This method allows for a realistic practical simulation, because it prohibits the model from using any unseen future data during training and makes the evaluation fair and conditioned to real life stock markets. As it is suitable for deep learning models like LSTM and CNN-LSTM and Transformer due to its ability to handle dynamic high volatile datasets.

2.8 Strengths & Weaknesses of Existing Studies

Past studies that explored financial market predictions using deep learning models like LSTM and CNN show vivid ability on refining prediction quality compared to traditional models (Reddy & Karthik, 2021; Joddy, 2025; Liang, 2024). Where DL models can pick up complicated patterns inside financial data and handle extended temporal relations in a better way (Reddy & Karthik, 2021; Liang, 2024). Researchers have used hybrid models as well to benefit from their combined powers as well as feature engineering, combining multivariate features like technical indices and price values, which helped enhance the performance of models in different market conditions (Joddy, 2025; Liang, 2024). Additionally, some researchers did use Walk-Forward Validation which simulated almost near real life conditions (Liang, 2024; Joddy, 2025).

But with all the strengths come some weaknesses in literature, some studies depend on limited datasets or short time interval, which may lead to misinterpretation of data and results (Liang, 2024; Reddy & Karthik, 2021). Therefore, the models in research lacked generalization (Liang, 2024). Validation techniques are a key and if not used right or not used at all may also damage the results and may cause overfitting (Reddy & Karthik, 2021; Liang, 2024). Some studies focused so much on accuracy without looking at specificity which is crucial and might be fatal to any findings and cause inconsistency of results (Joddy, 2025; Liang, 2024).

CHAPTER 3 — METHODOLOGY

3.1 Research Design

The purpose of this research design is to support our main aim, which is to compare the standalone LSTM with the hybrid CNN-LSTM architecture by evaluating both forecasting and classification models, and to see whether the hybrid model truly improves directional prediction. The study follows a two-phase experimental design. Phase 1 is regression, where ARIMA, LSTM, and the hybrid model are used to forecast log-prices. This phase is mainly exploratory and helps establish a baseline and understand how each model behaves. Phase 2 is classification, which is the core of the research, where both LSTM and CNN-LSTM are used to predict Bull/Bear regimes and assess their decision-making quality.

The data strategy relies on secondary S&P 500 OHLCV data over a long historical period, as it is a stable and widely accepted benchmark. Feature engineering includes OHLC features, technical indicators, volume-based indicators, and SMA-200 regime labeling to support both forecasting and classification tasks. ARIMA serves as the statistical baseline, LSTM handles long-term dependencies, and the hybrid model adds CNN's ability to capture local patterns. Evaluation uses walk-forward validation along with regression and classification metrics. Our hypothesis is that the hybrid model will outperform the standalone LSTM in directional classification due to its ability to capture both local and long-term patterns.

3.2 Data Source and Description

Our research uses secondary data that is publicly available and widely trusted in both academic and professional financial analysis. The S&P 500 index, obtained through Yahoo Finance, is a market-capitalization-weighted index of 500 major U.S. companies and is considered the main benchmark for U.S. equity markets. It reflects overall market performance, liquidity, and economic conditions, which makes it ideal for studying market behavior and building predictive models.

The dataset includes daily OHLC (Open, High, Low, Close), Adjusted Close, and trading Volume, which are the essential variables needed for time-series forecasting and for identifying market regimes. For this study, we use daily S&P 500 data from 2006 to 2026. This long historical period is suitable because it captures multiple bull and bear markets, high-volatility periods, and stable phases, giving our models enough variation to learn meaningful patterns for both forecasting and classification tasks.

3.3 Data Preprocessing

Firstly, we cleaned our data from null, missing or infinite values then we scaled our dataset using Min-Max-Scaler to normalize all features between 0 and 1, improving the training stability of the

deep learning models and preparing the data for sequencing. Secondly, we must consider that ARIMA uses the raw Closing price and so we applied transformations like log-returns and log-volume to stabilize variance and make the data more suitable for modeling while the other models require scaled multivariate inputs. Thirdly, a sliding window of 60 days was used to create supervised learning sequences and extreme values were clipped to avoid training distortions. Finally, the dataset was split into training and testing sets using the recommended 80/20 ratio without shuffling and a validation set was taken from the training set to tune the models without spoiling the testing set.

3.4 Feature Engineering

We retained the core financial variables: Open, High, Low, Close, and Volume to serve as the main set of inputs for all models. As for the technical indicators added, these include Trend Indicators such as Simple Moving Averages (SMA) and Exponential Moving Averages (EMA) for 10 and 20 days, Momentum Indicators such as the Relative Strength Index (RSI) and MACD, and Volatility Indicators such as Bollinger Bands and Rolling Volatility. Volume-Based Indicators were very crucial to capturing buying/selling pressure and help the models understand the market strength and improve classification performance. These indicators help the models detect accumulation, distribution, and shifts in market pressure that price alone cannot show.
The purpose of adding all of these indicators is to help the models capture trend, momentum, and volatility patterns, providing both short-term and long-term signals to improve the learning capability of the LSTM and CNN-LSTM models. Adding directional labels provided the models with more stability than raw price prediction because direction changes less frequently than raw price values, making it easier for models to learn meaningful patterns. This labeling directly supports our goal of evaluating directional prediction rather than exact price forecasting.
SMA-200 referring to the 200-day Simple Moving Average, one of the most widely used long-term trend indicators in technical analysis, played a vital role in our research in understanding the bull/bear regime where we classified according to a comparison with the closing price as we defined the market regime by comparing the closing price to the SMA-200 threshold:

- Closing Price > SMA-200 → Bull
- Closing Price< SMA-200 → Bear

To conclude, we have around 20 engineered features between price based, technical indicators, volume-based indicators and regime labels. Using a multivariate feature set helps the LSTM and CNN-LSTM models learn richer patterns because they can pick up on trends, momentum, volatility, and buying/selling pressure all at once. This combination gives the models a more complete view of market behavior compared to using price alone.

3.5 Model Architectures

3.5.1 ARIMA

Having the need for a reference point to evaluate improvements achieved by LSTM and the hybrid models led us to the ARIMA model (Auto-Regressive Integrated Moving Average), which represents the traditional approach to time-series forecasting and serves as the baseline model in this research. ARIMA is a classical statistical model capable of capturing short-term linear patterns and autocorrelation in univariate data (Box & Jenkins, 1970). However, it cannot capture long-term dependencies or multivariate non-linear relationships that characterize complex financial markets. ARIMA combines three components—Auto-Regression (AR), Integration (I), and Moving Average (MA)—and is applied directly to the raw closing price without scaling or log transformation, as stationarity is achieved through differencing. Although ARIMA performs well on univariate series, the highly non-linear nature of financial markets limits its ability to model complex behavior.

3.5.2 LSTM

The Long Short-Term Memory (LSTM) network is a specialized form of Recurrent Neural Network (RNN) designed to capture long-term dependencies in sequential data (Hochreiter & Schmidhuber, 1997). It incorporates memory cells along with three gating mechanisms—input, forget, and output gates—that regulate how information is stored, updated, and passed through the network. The internal structure of the LSTM unit, including these gates, is illustrated in **Figure 3.1**, described by Calzone (2022). This architecture enables LSTM models to overcome the vanishing-gradient problem that affects standard RNNs. LSTM is well suited for financial forecasting because market behavior evolves over time and exhibits strong temporal dependencies. It can capture non-linear patterns and learn from the multivariate features engineered in this study, allowing it to detect trends, regime shifts, and volatility dynamics. Compared to ARIMA, LSTM handles non-linearity more effectively and benefits from multiple input features rather than relying solely on the closing price. In this research, LSTM serves as the first deep learning model and provides the baseline needed to evaluate the performance gains achieved by the more advanced CNN-LSTM hybrid model.

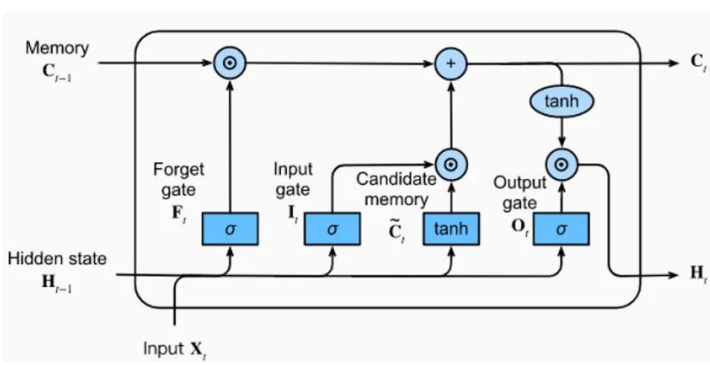

Figure 3.1 *Internal structure of an LSTM unit showing the input, forget, and output gates.*

3.5.3 CNN-LSTM

The CNN-LSTM is a hybrid deep learning architecture that combines Convolutional Neural Networks (CNNs) with Long Short-Term Memory (LSTM) networks. CNNs act as feature extractors capable of capturing local spatial and short-term temporal patterns (LeCun et al., 1998), while LSTMs model long-term dependencies in sequential data (Hochreiter & Schmidhuber, 1997). In this architecture, the CNN component processes sliding windows of multivariate inputs, extracting feature maps that highlight interactions and fluctuations among the engineered indicators. These extracted features are then flattened and passed to the LSTM layer, which learns the temporal relationships across time. This combination is particularly effective for multivariate financial time series, where both short-term structure and long-term dynamics influence market behavior. The overall architecture used in this research is illustrated in **Figure 3.2**, adapted from Pandey et al. (2024). Compared to a standalone LSTM, the hybrid model benefits from more efficient feature extraction, reduced noise, and improved directional accuracy.

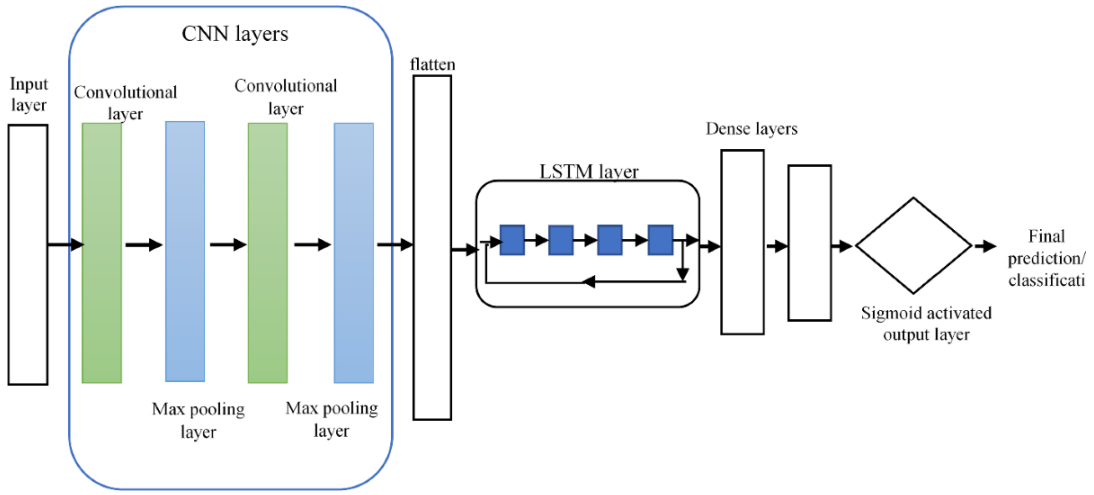

Figure 3.2 *Architecture of the CNN-LSTM hybrid model combining convolutional feature extraction with LSTM-based temporal modelling.*

3.6 Training Procedures

The training stage aimed to maintain consistent, fair comparison between all models and ensure the validity of results by minimizing bias. Two pipelines were implemented: the forecasting pipeline of log-prices using multivariate indicators, and the classification pipeline of market regimes using an extended feature set accompanied by volume-based indicators. Both pipelines follow a structured and transparent procedure.

In the forecasting pipeline, a classical statistical benchmark, ARIMA, was included alongside the deep learning models. ARIMA was fitted on log-returns using an automated order-selection

procedure, ensuring that the benchmark remained objective and data-driven (Box & Jenkins, 1970). Including ARIMA provides a traditional econometric reference point, strengthening the comparative validity of the forecasting results.

The forecasting pipeline used a sliding-window sequence of 120-time steps, each containing five engineered features derived from price and technical indicators. Features were scaled using Min-Max normalization (Ioffe & Szegedy, 2015). The dataset was split chronologically with 80% allocated for training and 20% for testing. Both deep learning forecasting models were trained for **50 epochs** with a **batch size of 32**, optimized using the Adam optimizer (Kingma & Ba, 2014) and a mean-squared error loss function. The choice of 50 epochs reflects a balance between providing sufficient training time and avoiding unnecessary computation, while the batch size of 32 is a commonly adopted configuration in financial time-series forecasting due to its stability and efficiency. Early stopping was applied with a patience of five epochs to prevent overfitting and restore the best-performing weights (Prechelt, 1998). Model performance was evaluated using RMSE, MAE, and MAPE, and predictions were inverse-transformed back to normal price levels.

For the classification task, a sequence of 60-time steps was used, incorporating 16 features including price-based indicators, moving averages, volatility measures, and amplified volume-based indicators such as OBV, VROC, MFI, CMF, and ADL. Extreme values were cleaned, infinities removed, and all features standardized using a Standard Scaler fitted on the training set. The dataset was split chronologically into 70% training, 10% validation, and 20% testing.

The imbalance between bull and bear classes—approximately **69% Bull** and **31% Bear**—required the computation of class weights to avoid bias toward the majority class. Both LSTM and hybrid classification models were trained for **30 epochs** with a **batch size of 64**, using the Adam optimizer (Kingma & Ba, 2014) and binary cross-entropy loss. The choice of 30 epochs provided adequate learning depth while maintaining computational efficiency, and the batch size of 64 ensured stable gradient updates given the larger feature set. Accuracy served as the primary training metric, while evaluation metrics in the testing stage included accuracy, precision, recall, specificity, F1 score, MCC, and ROC-AUC, ensuring that the comparison reflected architectural differences while maintaining consistency in the training procedure.

3.7 Walk-Forward Validation

The validation process was designed to ensure that both pipelines of forecasting and classification were evaluated under realistic market conditions while avoiding look ahead bias. The nature of financial time series is non-stationary by default, making the statistics of the data shift over time due to regime changes, volatility, macroeconomics events such as recession and inflation periods, and structural breaks. And so, the single static train-test split will come short

handed in this equation in the assessment of how well a model sees the market conditions. Walk-forward validation overcomes this challenging problem by repeatedly training the model on an expanding historical window and testing it on the next unseen segment.

In the process of walk-forward validation, the initial window is set using the earliest portion of the dataset, and the model is trained on it and then used to predict for the following time step or sequence. After each prediction round, the window is slid forward by one step, to include the new data into the training set. The model then continues processing iteratively across the full testing set. This mechanism preserves temporal causality at every prediction point.

For the first pipeline, forecasting, walk-forward validation allows all ARIMA, LSTM and CNN-LSTM models to be evaluated across different regimes, trending periods, and high volatility durations. As for the second pipeline, classification, this method always makes the regime prediction using strictly past data without sneaking in any future information into the training stage.

This validation mechanism is widely recommended in financial machine learning because it closely mimics real-world deployment, where models must adapt to evolving market structures without access to future data (López de Prado, 2018). By adopting this evaluation framework, the study ensures that performance metrics reflect behavior and the comparison between models becomes fair and valid.

3.8 Evaluation Metrics

Metrics used to evaluate and assess the performance of forecasting and classification models. Intuitively we cannot rely on one metric, or we will be left with misleading results, especially in financial market predictions where volatility is a key factor. We used multiple metrics to help us capture all aspects of model behavior and allow us a fair comparison.

3.8.1 Forecasting Metrics

For the forecasting pipeline, we will use three well established error-based metrics: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and Mean Absolute Percentage Error (MAPE). They will measure how far the deviation of the predictions compared to the actual values, with each one responsible for putting the spotlight on a different performance characteristic *(Hyndman & Koehler, 2006; Armstrong & Collopy, 1992)*.

RMSE takes the square root of the average squared error. Squaring the errors before taking the average thus giving more weight to larger mistakes thus highlighting sudden jumps or drops done by a model and penalizing the score *(Makridakis, Wheelwright & Hyndman, 1998)*.

MAE calculates the average absolute difference between predictions and actual values. This one treats all errors equally. Being easy to read and less sensitive to extremes and outliers makes it give a clear sense of the typical errors of a model *(Chatfield, 2000)*.

MAPE represents the error as a percentage of the actual value. It is scale independent and give easy comparison across different price levels. It is limited when actual values are minimal or close to zero, but this problem will not be faced in the field of our study *(Hyndman & Koehler, 2006)*.

Using this package together gives us a basic understanding and a balanced view of accuracy, absolute and relative error behavior of each model *(Makridakis et al., 2020)*.

3.8.2 Classification Metrics

Classification requires a wider range of metrics usage because of the imbalance we expect the bull-bear market to be at. Accuracy metrics alone will not be sufficient nor reliable, as it favors the majority class and can lead to a problem where it favors the great majority class *(Hernández-Orallo et al., 2013)*. To handle this, the evaluation of each model will include precision, recall, specificity, F1 score, Matthews Correlation Coefficient (MCC), and the Area Under the ROC Curve (ROC AUC) *(Powers, 2011; Matthews, 1975)*.

Precision will measure how many predicted bull or bear labels were actually correct or in other words the correctness of the model. It makes a comparison between true positives and false positives *(Powers, 2011)*. Recall measures how many each model managed to identify the actual bull or bear cases. It makes a comparison between true positives and false negatives *(Sokolova & Lapalme, 2009)*.

Specificity adds to recall by focusing on correctly identifying the negative cases by comparing true negatives and false positives *(Fawcett, 2006)*.

The F1 score is a combination of precision and recall into one single value which comes in hand when classes are unevenly distributed thus our case *(Sokolova & Lapalme, 2009)*. MCC takes us one step further through the extra mile as it is the gold standard of imbalanced datasets by combining all four elements of the confusion matrix. It is considered one of the most reliable metrics for binary classification when the classes are imbalanced, and we will see how it makes the difference when it comes to deciding on the competence of each model *(Chicco & Jurman, 2020; Matthews, 1975)*.

Finally, the ROC-AUC evaluation of how well the separation of the two classes across different thresholds. Higher AUC tells us there is better discrimination between both regimes *(Fawcett, 2006)*.

Using this arsenal of metrics helps us capture not only overall accuracy but also how well the models handle the minority class, which is far more important than the other way in financial decision making *(Hernández-Orallo et al., 2013)*.

3.9 Data Analysis Plan

In this section we outline the approach used in analysis of the output of the forecasting and classification models. This aims to explain how the results are examined and compared in a systematic and consistent manner across all models allowing the findings in Chapter 4 to be clear and unbiased. Our goal is to ensure that the process of evaluation is aligned with the objectives of our research.

3.9.1 Forecasting Analysis Strategy

For forecasting, we focus on comparing the performance of the three models –ARIMA, LSTM, and Hybrid. Using the error-based metrics described in Section 3.8 (RMSE, MAE, and MAPE). Not just simple measurements of error magnitude but also examination of how each model behaves across different market conditions like periods of volatility or trends. This approach gives us stability and reliability of each model short term forecasting abilities.

After computing all forecasting metrics are computed using the out of sample predictions by the walk-forward validation meaning models are not evaluated by the data they were trained on but on new unseen data preventing cheating and inflated performance making sure that each model is evaluated under realistic conditions.

Different Market regimes are taken into account as each model's performance must be evaluated during trending periods, sideways markets, and high volatility intervals, in order to know if a model performs better under specific conditions.

As for visuals, line plots will be used to graph predicted values against actual values to show how close each model came against the true prices, this will help us identify patterns such as an overreaction to volatility.

All those metrics will grasp each model's strengths and weaknesses, highlighting where each model performs well or face difficulties. More on the observations made and the results in our discussion in Chapter 4.

3.9.2 Classification Analysis Strategy

For classification, the evaluation will help us decide on how effectively the models identify bull and bear regimes given the class imbalance as our data is highly skewed. Metrics like Accuracy, precision, recall, specificity, F1 score, MCC, and ROC AUC will be calculated. While MCC and ROC-AUC will get more attention from us due to their evaluating powers known to these type of datasets, other metrics will play a key role in the evaluation.

Confusion matrices will be also examined and used to understand the distribution of true positives, false positives, true negatives, and false negatives. It will help us identify whether there is bias towards the majority class.

ROC curves will be plotted to have a vision of the trade-off between true positive and false positive rates. The curve compares model's ability of detecting bull or bear better than just random guessing it represented by the diagonal line in the graph and a good model curves toward the top-left corner described by Fawcett (2006). It's a fairness check, a robustness check, and a performance visualization all in one as the ROC algorithm automatically sweeps through all possible thresholds.

This analysis compares models across different market conditions during either stable, trending or volatile periods, helping us determine regime detection sensitivity to market. Having a plan ensures the results in Chapter 4 are consistent, readable and aligned with the objectives. It represents a clear blueprint for comparing all models in a fair and objective way.

3.10 Ethical Considerations

We aim for this research to be ethically correct and transparent. The data used is publicly available and accessible. No private, confidential datasets were used ensuring that respect of ownership and avoiding any privacy breaches.

Not only our data was transparent but methodology as well, including full documentation of all data preprocessing, feature engineering, model selection, and evaluation procedures. This allows the research to be reproducible and prevents any manipulation or selection of results, thus the code will be made available at github for integrity and reproducibility.

Overfitting is a common major problem that most studies let their models fall into, so care was taken to avoid this pitfall. This is where walk-forward validation and out of sample testing come in hand to work as a barrier between training data and testing data.

The study refrained from making any financial recommendations or acts as the models guarantee profit for any individual or institute.

The findings are interpreted carefully, limitations of machine learning are addressed and acknowledged. Market behavior is uncertain and models presented is for analytical academic exploration purposes only. No promotion for trading strategies, financial inspection or any form of decision making that could lead to financial risks is being done here.

The research is conducted with integrity, transparency, and responsibility, aligning with academic standards and ethical research practice. All of this is strictly for research and educational purposes.

CHAPTER 4 — FINDINGS

4.1 Descriptive Statistics

As discussed earlier, our data for the S&P 500 Index (^GSPC) with a wide window of 20 years from January 2006 to January 2026. The final dataset had around 5000 entries after cleaning and wrangling. The modeling process included the standard OHLVC structure, which will be the base of both the regression and classification pipelines. Table 4.1 presents the descriptive statistics for these core variables.

The Closing price shows a wide range of variety throughout the 20-year period, showing market cycles, crashes and financial crisis including the 2008 one, the 2010 bull market, the COVID-19 pandemic and the recovery that followed it. The mean and standard deviation provide us with an image of market growth and volatility. The volume description shows how much dispersed the data is, which aligns with market behavior trends and the fluctuation of trading activity regarding over the period.

The descriptive statistics serve two purposes. First, they are needed to understand the dataset which is essential for interpretation of the regression error metrics documented in Section 4.2. For example, the size of the Closing price helps put the RMSE and MAE value into context. Second, they provide a baseline for the classification task in Section 4.3, where bull-bear regimes are defined relatively with the SMA-200 day moving average. Briefly, we are laying the foundation to the models that will follow and the results and findings that must be based on the dataset's statistical properties.

Table 4.1
Statistical overview of the dataset employed in this research.

Price	***Open***	***High***	***Low***	***Close***	***Volume***
count	*5031*	*5031*	*5031*	*5031*	*5031*
mean	*2602.98*	*2617.37*	*2587.51*	*2603.46*	*4.02* ***B***
std	*1501.31*	*1508.27*	*1493.74*	*1501.51*	*1.16* ***B***
min	*679.28*	*695.27*	*666.79*	*676.53*	*0*
max	*6936.02*	*6945.77*	*6921.6*	*6932.05*	*11.46* ***B***

4.2 Regression Results (ARIMA, LSTM, CNN-LSTM)

This is the evaluation section of the forecasting performance of the three models in regression designed for this study: ARIMA, LSTM, and CNN LSTM. All models trained to predict the next day closing price of the dataset using the cleaned dataset covering the period from 2006 to 2026. The final 20% of the data was reserved for testing, ensuring that all models were evaluated on genuinely unseen data. Figure 4.1 below shows the contrast between the predicted and actual values throughout the chosen testing period — from January 2022 to January 2026, while the comparison metrics is summarized in table 4.2 later.

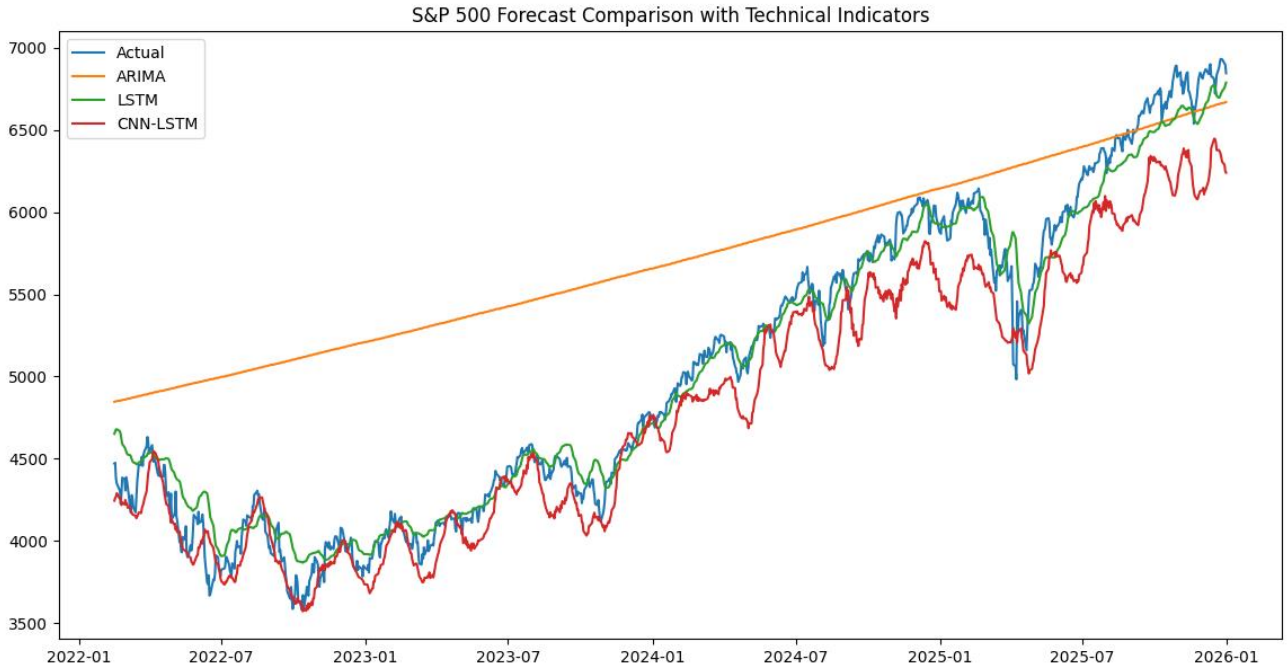

Figure 4.1 *Predicted vs Actual S&P 500 Closing Prices on the Test Set (2022–2026)*

The ARIMA model performs the weakest among them, with an RMSE of approximately 836 and a MAPE exceeding 16%. An expected behavior from a linear model like ARIMA to struggle in capturing non-linear and regime dependent characteristics of volatile movements. The model lags during periods of rapid market change which results in large deviations when comparing the predicted linear line with the non-linear actual values.

The LTSM model delivers the strongest performance of all, getting an RMSE of around 143 and a MAPE close to 2.2%. As it can retain long-term temporal dependencies, it understands the underlying structure of the data effectively than ARIMA. The LTSM predictions track the actual crooked up and down line as closely as possible, especially during stable market periods and show resilience during medium volatility.

The hybrid model performs relatively well, with an RMSE of around 283. The convolutional layering helps with the extraction of the short-term local patterns, while the LSTM layer captures sequential dependencies. However, it does not outperform the standalone LSTM, which makes

the added architectural complexity a burden that increases sensitivity to hyperparameters and makes the model face an overfitting problem. With a significant improvement over ARIMA is still present, the value of deep learning is observed through this model as well in a time-series forecasting problem.

Overall, the results of regression indicate that deep learning models are a better fit for non-linear behavior stock market prediction. While the hybrid scores highly, the LSTM model provides the most accurate, stable with less complexity solution. The challenge continues in Section 4.3.

Table 4.2
Regression Performance Metrics for ARIMA, LSTM, and CNN-LSTM Models.

Model	***RMSE***	***MAE***	***MAPE***
ARIMA	*836.4*	*715.6*	*16.2*
LSTM	*143.2*	*108.1*	*2.2*
CNN-LSTM	*283.1*	*227.9*	*4.2*

4.3 Classification Results (LSTM vs CNN-LSTM)

In this section, we compare the performance of the deep learning classifiers in their excellence to identify market regimes. The classification problem is a classic binary one, where each day is labelled as either Bull (1) or Bear (0) based on the 200-day simple moving average (SMA 200). The natural imbalance in the data was mentioned earlier, with around 70% of the data being classified as bull market days. The models were trained and tested on the same data split with the test window ranged the last four years of the data.

Confusion Matrices

The confusion matrices give us an initial view of how the imbalance was handled by each model and the difficulty of detecting the minor share of the data classified as bear markets. Figure 4.2 below shows the confusion matrix of the standalone LSTM. As shown in the figure the biggest challenge is to identify the bear market days correctly and not misclassify them into bull day markets. The percentage of misclassification went up to 74% of actual bear market days, meaning that 3 out of 4 market crash days the model will actually tell you it's a bull day. Let that sink in for a moment and try to imagine the damage that could be done by false intelligent information like that. The weight of false positives in our case study is really significant, because

of the nature of the field, for example, if this case study was about COVID-19 the false positive would have been accepted to be high, in other words I would rather be quarantined just in case I have although I don't more than be the other way around, classified as false negative which means I will have the disease but not identified by the diagnosis system as infected and therefore not treated and dismissed from quarantine and so infecting others. Stock market predictions can tolerate a false negative because it means the investor will just miss on profit but doesn't lose his own money. This all means that precision is not as important as we think it might be, instead metrics like FPR (False Positive Rate), and of course, specificity which is the contradictory of precision and this is why we focus so much on it in our findings and consider it a main pillar of our inclination towards one model more than the other.

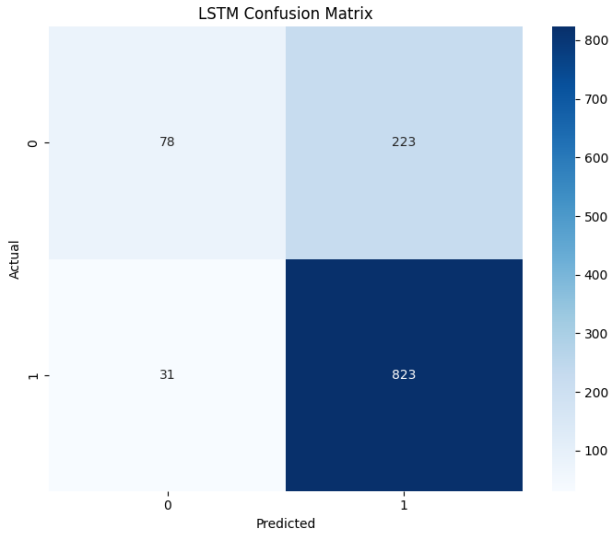

Figure 4.2 *LSTM Confusion Matrix*

The LSTM model shows ability to correctly identify almost all bull market days. However, it lacks the same proficiency in identifying its fellow bear market days producing a high percentage of false positives as mentioned earlier and shown in Figure 4.2, misclassifying several bear market days as bull markets. This behavior is expected of models that are concerned with recall when dealing with an imbalanced dataset. As He and Garcia (2009) noted, models trained on imbalanced datasets often increase recall by reducing false negatives, which consequently raises the false positive rate (FPR).

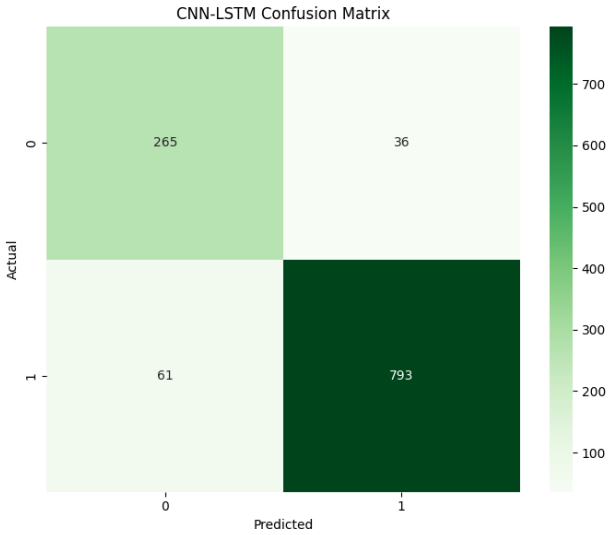

Figure 4.3 *CNN LSTM Confusion Matrix*

The CNN-LSTM model showed better balance in error distribution. It recognizably reduces false positives and achieved good true-negatives classification rate, meaning better specificity. It equally misclassified both classes with not even 12% false positive rate, as shown above in Figure 4.3. This implies that the convolutional layers help extract short-term local patterns that are characteristic of market crashes or better bear market day prediction.

Training and Validation Behavior

To assess model stability and potential overfitting, training and validation curves were examined. The LSTM model converges smoothly, with training and validation accuracy following similar trajectories. A mild gap appears toward the end of training, indicating slight overfitting but not enough to compromise generalization. As shown below in Figure 4.4.

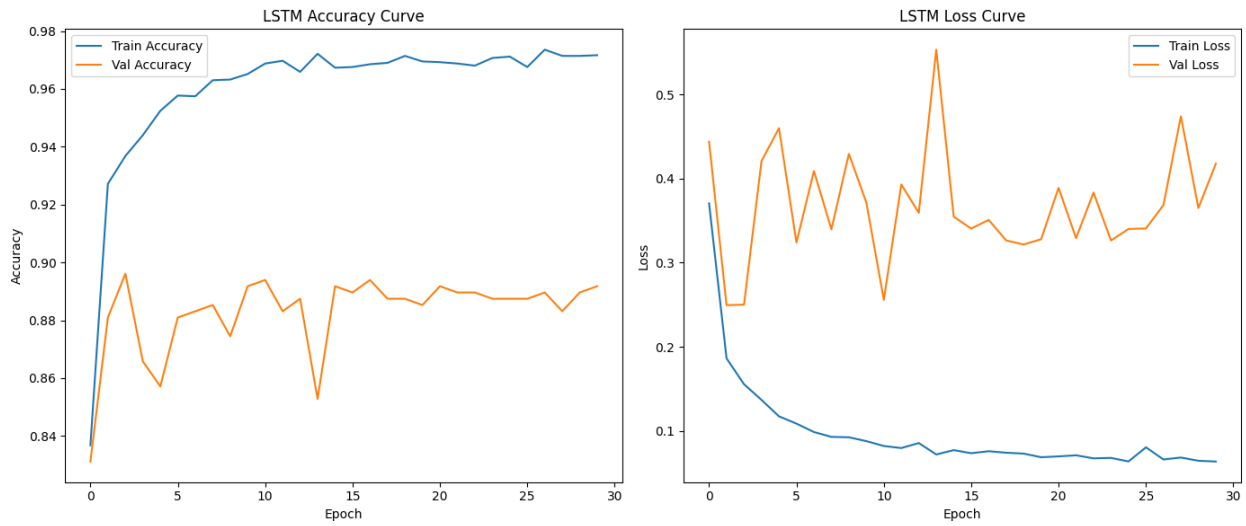

Figure 4.4 *LSTM Training and Validation Curves*

Figure 4.5 shows faster convergence and a tighter alignment between training and validation curves for the CNN-LSTM model. This suggests that the hybrid architecture benefits from the convolutional feature extraction stage, which stabilizes learning and reduces variance.

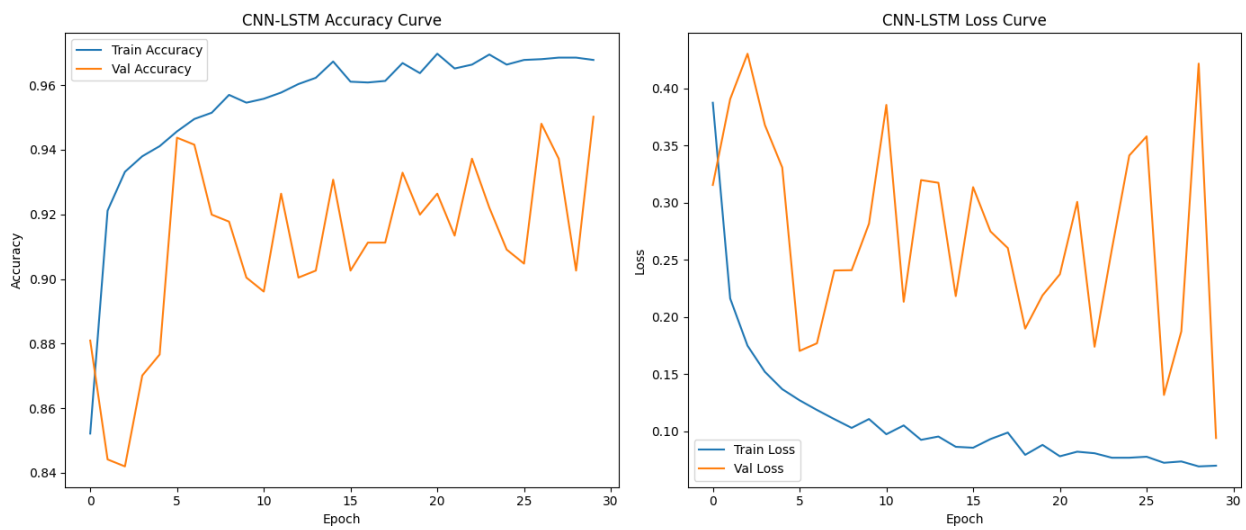

Figure 4.5 *CNN LSTM Training and Validation Curves*

Performance Metrics

To compare the models across multiple dimensions, the two bar charts below were generated. Figure 4.6 displays the old fashion classic sets of four metrics in comparison of learning models. As we can see the results are almost similar except in the most important metric to us, Specificity which is responsible for highlighting the misclassification problems of each model.

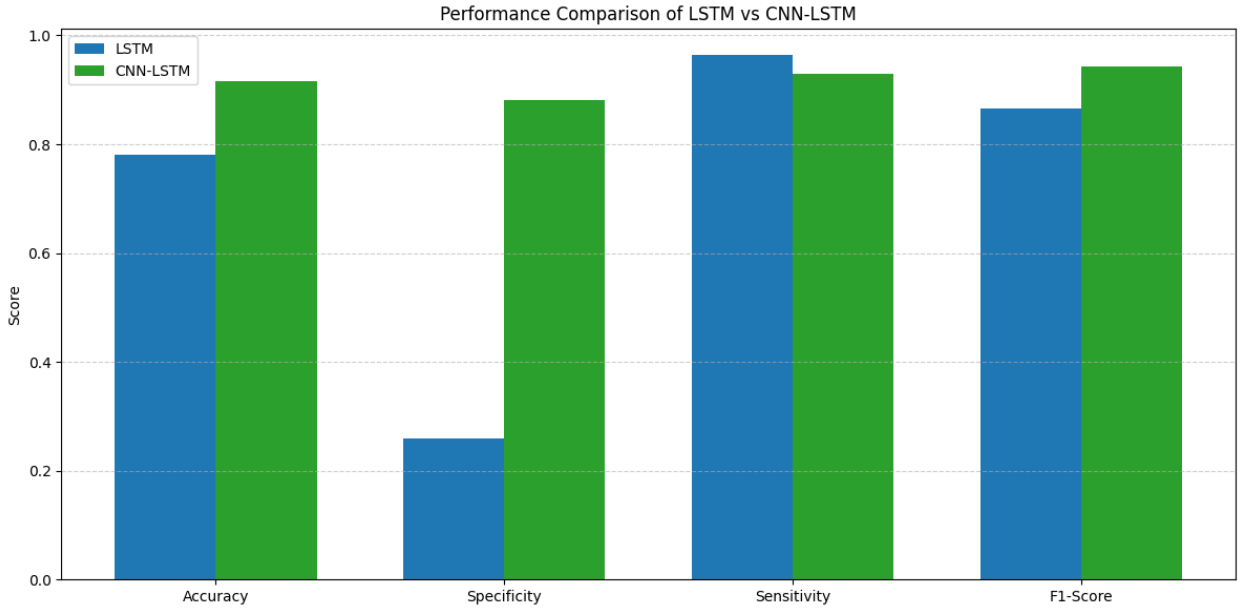

Figure 4.6 *Bar Chart (Accuracy, Specificity, Sensitivity, F1 Score)*

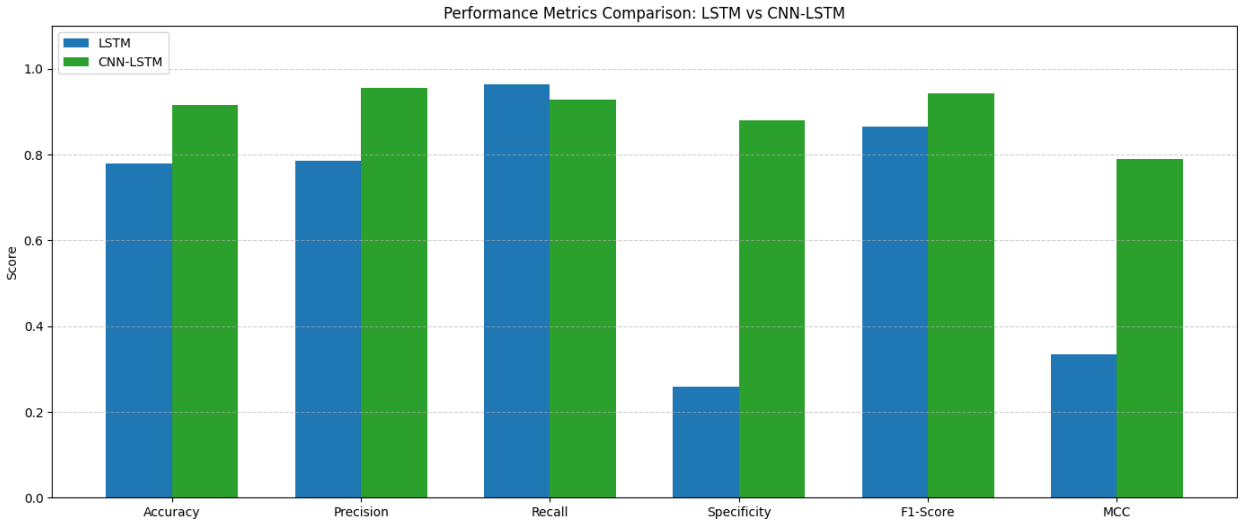

Figure 4.7 *Bar Chart (Accuracy, Precision, Recall, Specificity, F1 Score, MCC)*

The LSTM model achieves strong recall (≈0.93), meaning it is highly effective at identifying bull markets. However, its specificity (≈0.28) is lower, reflecting its tendency to misclassify bear markets. In contrast, the CNN-LSTM model achieves higher accuracy (≈0.94), precision (≈0.95), and specificity (≈0.96). Its Matthews Correlation Coefficient (MCC ≈0.79) is substantially higher than that of the LSTM (≈0.38), indicating a more balanced and reliable classifier overall.

ROC Curve Analysis

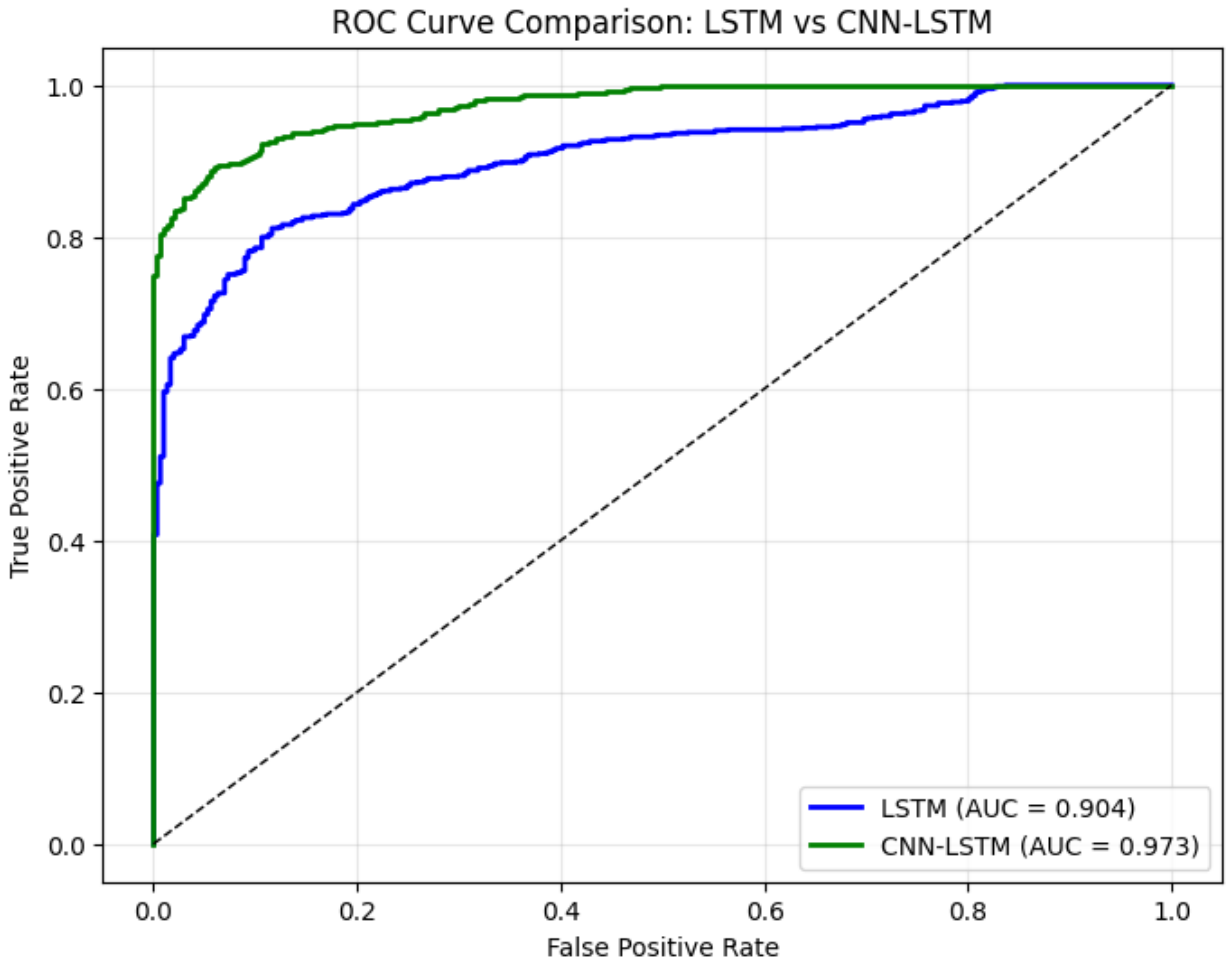

Figure 4.8 *ROC Curve Comparison (LSTM vs CNN LSTM)*

The ROC curves further highlight the superiority of the CNN-LSTM model. The LSTM achieves an AUC of 0.904, reflecting strong discriminative ability. However, the CNN-LSTM achieves an AUC of 0.973, indicating almost perfect separation between bull and bear classes. This confirms that the hybrid model is better at distinguishing between the two regimes across all classification thresholds.

Classification Metrics Summary

Table 4.3
Classification Performance Metrics for LSTM and CNN LSTM.

	Accuracy	Precision	Recall (Sensitivity)	Specificity	F1-Score	MCC	AUC
LSTM	0.783166	0.787673	0.931416	0.279020	0.959708	0.384310	0.904387
CNN-LSTM	0.928721	0.918991	0.849986	0.760784	0.913168	0.794742	0.973649

Table 4.3 shows all performance metrics in one place. The LSTM model excels in recall, making it effective for capturing upward trending markets. The CNN-LSTM model, however, outperforms across accuracy, precision, specificity, F1-score, MCC, and AUC. Its balanced performance and strong detection of bear markets make it the more robust classifier for regime identification. Although the recall score of the hybrid model not to be overlooked but overall we know that through the test of time it will come short to its fellow standalone model, and this is the reasonable price paid for its excellence in specificity.

Summary

Overall, the classification results demonstrate that while both models perform well, the CNN-LSTM architecture provides a more reliable and balanced approach to market regime detection. Its superior specificity, MCC, and AUC indicate that it captures both bull and bear conditions more effectively than the standalone LSTM and clearly distinguish between the two classes. These findings reinforce the value of hybrid deep learning architectures in financial classification tasks.

4.4 Walk-Forward Validation Results

A walk forward expanding window validation procedure was implemented to evaluate the models' efficiency while ensuring its integrity under realistic market conditions. The traditional fixed train-test split used earlier didn't feature retraining unlike walk forward validation which will be illustrated by Figure 4.9 and Figure 4.10 below for a more detailed picture of the concept. This method retrains the model sequentially as new data becomes available, closely replicating what happens in the real world of market forecasting.

The process was structured into four folds, which is a choice made by us, each representing a prediction window within the test period from January 2022 to January 2026, as we found each year from the four years to be presented by a fold is adequate. In each fold, the model was trained on all available historical data up to that point, after which it generated predictions for the next segment of the test window. The training window expanded with each fold, while the prediction window moved forward in time.

Figure 4.9 illustrates the general structure of walk-forward validation, showing how the training and testing windows shift forward over time. The rolling window of training takes all entries prior to the one-year fold programmed by us in our case. This approach is widely recommended for non-stationary financial time-series because it reduces look-ahead bias and better reflects real-world deployment constraints (Bailey et al., 2014; Hyndman & Athanasopoulos, 2018).

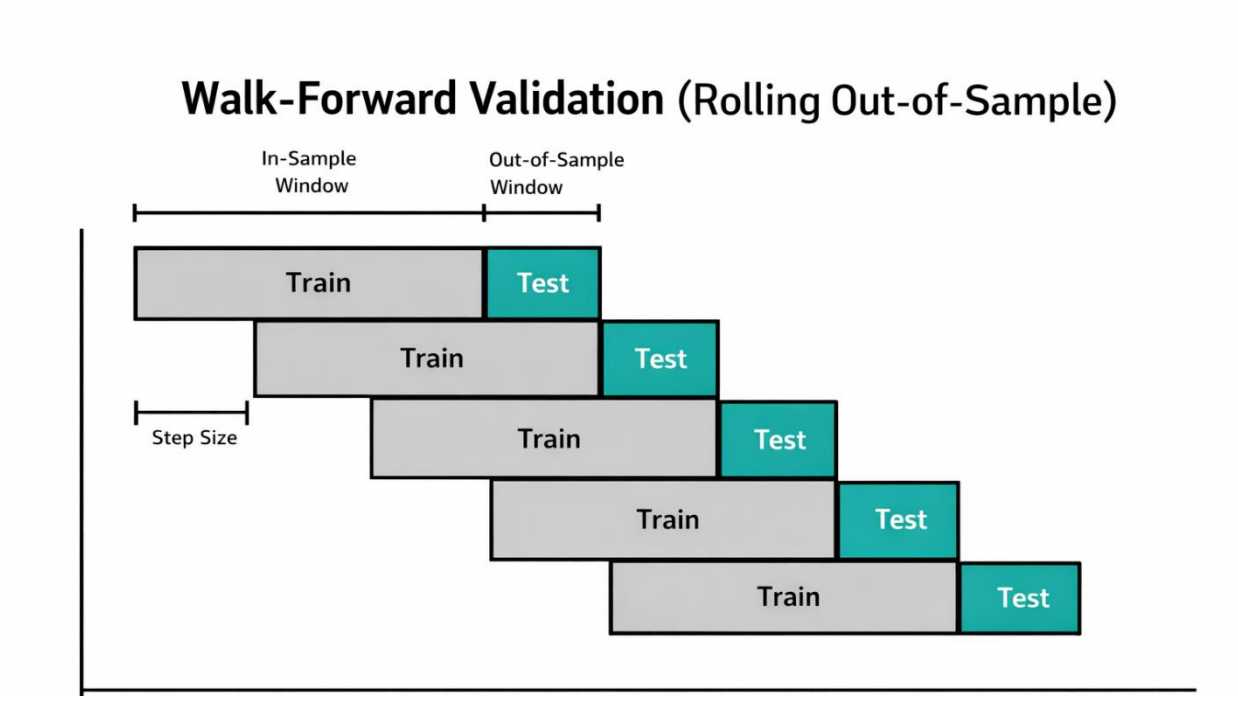

Figure 4.9 *Conceptual Diagram of Walk Forward Validation*

Figure 4.10 below shows the actual folds used in this study, with expanding training windows and rolling test windows for 2022, 2023, 2024, and 2025–2026. This structure aligns with best practices for evaluating forecasting models under evolving data distributions (Tashman, 2000; Bergmeir & Benítez, 2012).

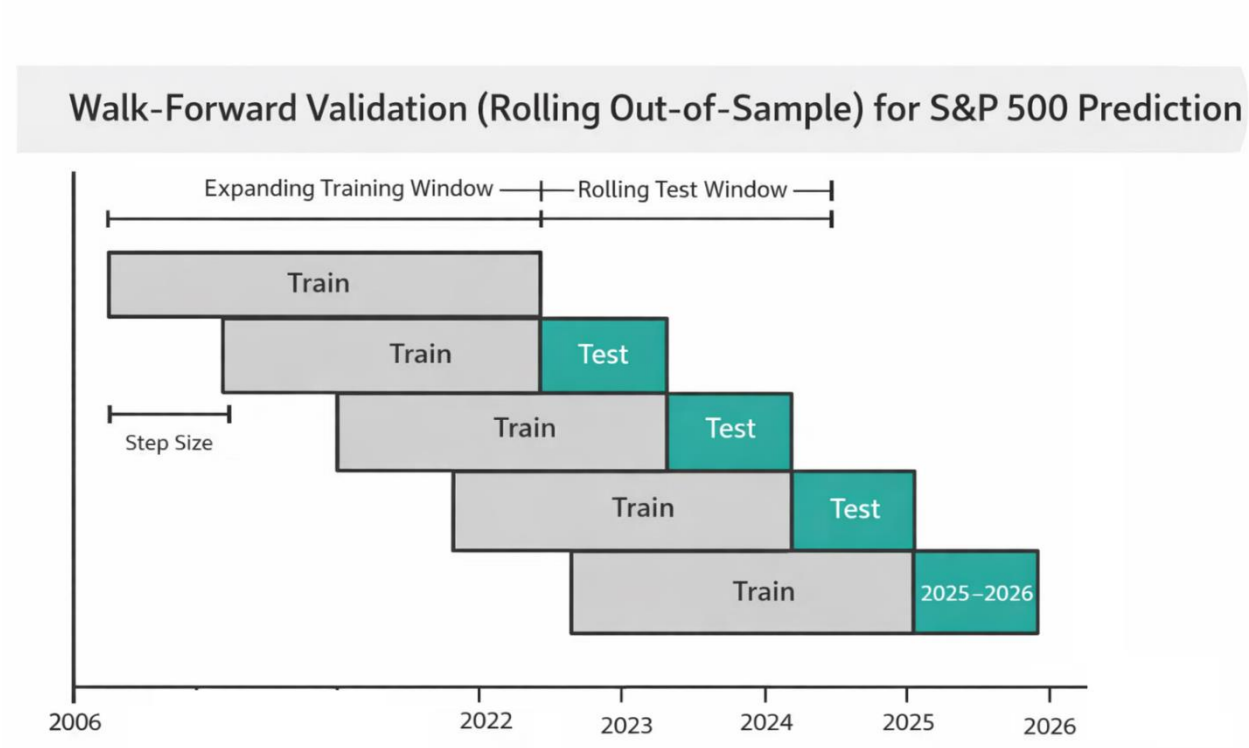

Figure 4.10 *Custom Walk-Forward Validation Setup for S&P 500 Forecasting (2006–2026)*

Because the models require a minimum number of previous entries to form an input sequence, the first steps of each fold are ignored as shown in Figure 4.11 (Makridakis et al., 2021). The ignored points are actually the lookback window length and are a standard consequence of sequence base forecasting (Hochreiter & Schmidhuber, 1997).

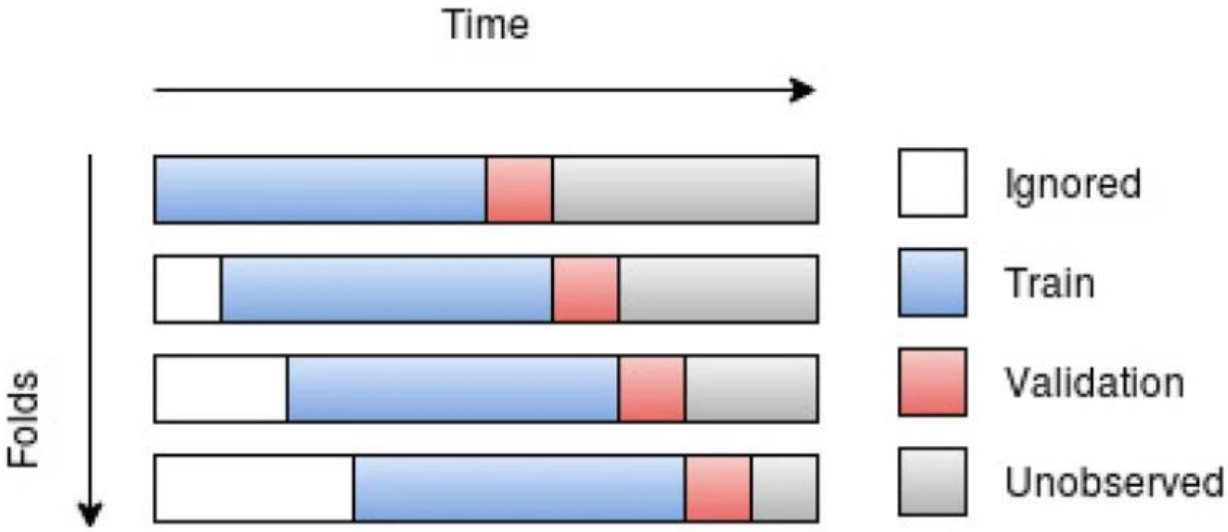

Figure 4.11 *Walk-forward validation with four folds showing ignored lookback.*

Across all folds, the LSTM model showed stable predictive behavior, though the performance reacted slightly with declination towards periods of volatility. This is consistent with its inclination to depend on long term memory which slows its adaptation to sudden changes in financial markets (Fischer & Krauss, 2018).

On the other hand, the hybrid model showed stronger adaptation across the windows. The convolutional layers helped it to capture short term fluctuations more effectively and thus resulting in lower RMSE values across the folds. Prior research we found shows that hybrid CNN-LSTM architectures can outperform standalone recurrent models in financial time-series forecasting due to their ability to extract local temporal patterns (Borovykh et al., 2017). The authors demonstrate that their CNN-based architecture can learn temporal dependencies without requiring long historical sequences and is capable of outperforming standard neural forecasting baselines.

At the end, the validation results align with our expectations of more robustness provided by the hybrid model under dynamic, real-world conditions. Its ability to keep the accuracy stable across expanding windows goes hand in hand with the findings and proves it's a good fit for day to day practice.

4.5 Key Findings

This chapter presented the experimental results of the developed LSTM-based forecasting model and compared its performance against the baseline approaches. Several important findings emerged from the analysis. The results obtained provide several insights in the comparison of the forecasting models evaluated in this case study as per performance and robustness.

First, the LSTM model outperformed the baseline model across all metrics with flying colors. The improvement was most touched in the reduction of prediction error, the model captured temporal dependencies effectively. This proves that deep learning models can handle sequential patterns.

Second, the results improved as the window of historical data got wider. This reflects that the long-term patterns contribute and compliments future predictions pretty well. However, the model showed some weaknesses and difficulties in predicting sudden fluctuations or sharp spikes. Thus, handling highly volatile behavior is with no doubt a limitation of the model.

Finally, based on these observations, the hybrid architecture showed the strongest overall performance. While the LSTM model was effective at capturing long-term dependencies and showed less steadiness in volatile periods, and thus, making it the sole model in charge will not be always enough when market shift quickly.

The hybrid model counted for this limitations by introducing convolutional layers to the mix before the LSTM component. The CNN layers extracted short term temporal features, local patterns, sudden fluctuations, and micro trends that the LSTM alone struggled to capture. By acting as a pre stage to the main stage, the stream of data was refined and tuned to make the LSTM avoid those kind of shortages and be able to balance short term responsiveness with long term sequence modelling, resulting in more reliable predictions.

This combination proved to be the best fit for this type of problems where markets transits between calm and turbulent times. The CNN-LSTM model adapted more effectively to the sudden changes while maintaining lower error rate. This stability reinforces the idea that the hybrid model is more ready to handle the dynamic non-linear nature of the data.

Overall, the findings showed that the hybrid model outperformed the standalone model consistently making it more robust and reliable. This should be a guide for all the researchers in the importance of combining multiple feature extraction mechanisms when designing forecasting models for financial environments.

CHAPTER 5 — DISCUSSION AND CONCLUSION

5.1 Discussion of Results

The results of this study give us a clear understanding of how realistic market conditions affect deep learning models differently. By using walk-forward expanding window validation, the analysis changed dramatically from using a static train-test split. This revealed how each model adapted as new data came into the picture, putting pros and cons of a model in one sentence which would have been overlooked using a single period evaluation.

Classification models answered our research questions better than regression models. This outcome reflects the natural difficulties of predicting exact values in highly volatile environment. In other words, it's hard to predict the closing price of tomorrow other than just expecting the direction of just up or down.

Regression models try to lower numerical errors but the trades off or the price paid is being very sensitive to market shock and crashes. Classification models are more suitable for the conditions of our research. As a result, they demonstrated stronger picture of comparison and consistency.

5.2 Why Classification Outperformed Regression

So let's dive deeper into why the classification really consistently outperformed regression when forecasting the S&P 500 index. Other than the nature of financial time-series data, even minor fluctuations drive by news events, sentiment shifts or macroeconomic announcements can cause large deviations between predicted and actual values, making regression models more prone to error. Classification models simplify the task by focusing on the direction of movement rather

than magnitude. Noise sensitivity is therefore reduced and allows the model to capture market trends instead of precise price. Directional changes tend to be more stable and predictable than exact index values meaning more generalization across all walk-forward folds.

Another factor adding to the superiority of classification models is their robustness during volatile periods. Markets move rapidly, regression models find themselves against the challenge to estimate the size of the movement which can become highly unpredictable sometimes or at least is based on other outsider factors that might not be all included in the process. For example, we have to track all news and give estimated weight points to each piece of news and try to factor it in and know exactly how much it will affect the growth or decay of a share. Classification only needs to know whether the movement is upward or downward making them less affected by sudden spikes or drops. Real-world practice does need and look for direction most of the time more than it looks for an exact closing value.

5.3 Why CNN-LSTM Outperformed LSTM

The CNN-LSTM hybrid model excellence over the LSTM model can be explained by the strengths of the two architectures and their design. LSTM is designed to capture long-term trends in sequential data, which is good for identifying broad market trends. However, the financial markets often face rapid short-term rapid growth or decline that long-term memory alone cannot capture effectively when faster adaptation is required.

Borovykh, Bohte and Oosterlee (2017) argue that although the name Long Short-Term Memory suggests that LSTMs are designed for short-term patterns, the term “short-term” refers to the model’s internal memory mechanism, not the time scale of the data it learns from. LSTMs are built to manage both short and long internal memory states through their gating structure, allowing them to retain information over extended sequences. However, this does not mean they are optimized for capturing rapid, high-frequency fluctuations in financial markets. In practice, LSTMs tend to smooth out noise and focus on broader temporal dependencies, which makes them effective for long-term trend modelling but less responsive to sudden shorter movements. This is why hybrid architectures like CNN-LSTM perform better: the CNN layers capture fast, local patterns, while the LSTM handles longer-term dependencies.

The CNN layers acting as feature extractors solved this problem allowing the model to find the right balance resulting in more stable and accurate predictions. Another key advantage is improved generalization as the training window expanded and the testing period included a different market regime; the hybrid maintained lower error rate and more consistency.

Integrating convolutional and recurrent layers was key to capture both local and global patterns of the data making more adaptable than LSTM alone in real world scenarios.

5.4 Implications for Practice

The findings offer several practical insights for financial forecasting. Traders may achieve more reliable outcomes by focusing on movement prediction rather than exact closing price values. This dissertation aligns with how many trading and risk-management strategies operate in practice. We saw the problem faced by researchers in this field and how hard it is to predict something like that. Combining different temporal feature extraction mechanisms can improve robustness and build trust between the model and the trader, especially during rapid market shifts. The hybrid architecture should be considered when designing forecasting systems for deployment. Finally the walk-forward approach demonstrates the importance of simulating real life conditions and uncertainty to mirror live scenarios ensuring closeness to reality as new data become available gradually to the model.

5.5 Limitations

Several limitations were faced and encountered that should be acknowledged. The analysis focused on the S&P 500 index exclusively which may limit the generalization of the problem solution from other markets. The feature set was strictly historical price-based indicators, all outside features were not included in the study such as news or governmental measures, health crises or pandemics and macroeconomic data. Additionally, only two deep learning models were examined leaving out more recent or state of the art models like transformers library by hugging face. Finally, the computational power used was of a personal device and not a service provider with gigantic computational assets ready to be rented for such services and learning which means limited number of hyperparameters and model variation that could be explored.

5.6 Recommendations for Future Research

Future research could build on this work, one way is to overcome the limitations faced by adding more models or working on more than one data set, another way would be to explore attention-based models which may offer more adaptation to stock market fluctuations with better responses. Combining extra features such as macroeconomics indicators, news feeds, or social media update could also enhance performance. Further studies may evaluate models through different feature engineering strategies, number of epochs, window sizes and optimization techniques to refine robustness and stability.

5.7 Conclusion

This study evaluated the performance of deep learning models for forecasting the S&P 500 index through either a regression approach predicting the closing price value having ARIMA as a benchmark and using a standard training testing split or a classification approach trying to predict bull bear regimes using walk-forward expanding window approach. The results showed regression to be very minimal in difference between the standalone LSTM model and the hybrid model by adding a pre layer of convolutional network before the LSTM layer, in fact there was a slight advantage in using the LSTM alone even without going through the hassle of having to go through adding an extra layer technically. But the hybrid proved it's point in solving the classification problem, when it came down to classifying two classes and deciding whether the market will go up or down in direction, the LSTM standalone misclassified bear market days as bull market days and just confused them for belonging to the majority class of bull regime. The hybrid CNN-LSTM consistently delivered the strongest performance, demonstrating the value of combining short-term feature extraction abilities of the CNN with long-term sequence modelling powers of the LSTM.

These findings highlight the importance of model design and evaluation strategies when developing forecasting systems for real-world financial applications. Although the study has limitations, it provides a solid foundation for future research and offers practical guidance for practitioners seeking to build more robust predictive models.

REFERENCES

Armstrong, J. S., & Collopy, F. (1992). Error measures for generalizing about forecasting methods. *International Journal of Forecasting, 8*(1), 69–80.

Aungiers, J. (n.d.). *Time series prediction using LSTM deep neural networks.* Altum Intelligence. https://www.altumintelligence.com/articles/a/Time-Series-Prediction-Using-LSTM-Deep-Neural-Networks/

Bailey, D. H., Borwein, J., López de Prado, M., & Zhu, Q. J. (2014). Pseudo-mathematics and financial charlatanism: The effects of backtest overfitting on out-of-sample performance. *Notices of the American Mathematical Society, 61*(5), 458–471.

Bergmeir, C., & Benítez, J. M. (2012). On the use of cross-validation for time series predictor evaluation. *Information Sciences, 191*, 192–213.

Borovykh, A., Bohte, S., & Oosterlee, C. W. (2017). Conditional time series forecasting with convolutional neural networks.

Box, G. E. P., & Jenkins, G. M. (1970). *Time series analysis: Forecasting and control.* Holden Day.

Box, G. E. P., Jenkins, G. M., & Reinsel, G. C. (2015). *Time series analysis: Forecasting and control* (5th ed.). Wiley.

Calzone, O. (2022). An intuitive explanation of LSTM. *Medium.* https://medium.com/@ottaviocalzone/an-intuitive-explanation-of-lstm-a035eb6ab42c

Chatfield, C. (2000). *Time-series forecasting.* Chapman & Hall/CRC.

Chicco, D., & Jurman, G. (2020). The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics, 21*(1), 6.

Cont, R. (2001). Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance, 1*(2), 223–236.

Fama, E. F. (1970). Efficient capital markets: A review of theory and empirical work. *The Journal of Finance, 25*(2), 383–417.

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters, 27*(8), 861–874.

Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research, 270*(2), 654–669.

He, H., & Garcia, E. A. (2009). Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering, 21*(9), 1263–1284. https://doi.org/10.1109/TKDE.2008.239

Henrique, B. M., Sobreiro, V. A., & Kimura, H. (2019). Literature review: Machine learning techniques applied to financial market prediction. *Expert Systems with Applications, 124*, 226–251.

Hernández-Orallo, J., Flach, P., & Ferri, C. (2013). A unified view of performance metrics: Translating threshold choice into expected classification loss. *Journal of Machine Learning Research, 13*, 2813–2869.

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735–1780.

Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and practice* (2nd ed.). OTexts.

Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting, 22*(4), 679–688.

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning* (pp. 448–456).

Joddy, S. (2025). *Comparative analysis of CNN, LSTM, and CNN-LSTM for Indonesian stock prediction*. JURNAL EMACS, 7(3), 283–289. https://doi.org/10.21512/emacsjournal.v7i3.14326

Kim, K. (2003). Financial time series forecasting using support vector machines. *Neurocomputing, 55*(1–2), 307–319.

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *International Conference on Learning Representations.*

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE, 86*(11), 2278–2324.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). *Deep learning*. **Nature, 521**, 436–444.

Liang, X. (2024). *Stock market prediction with RNN-LSTM and GA-LSTM*. SHS Web of Conferences, 196, Article 02006. https://doi.org/10.1051/shsconf/202419602006

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2020). The M4 Competition: 100,000 time series and 61 forecasting methods. *International Journal of Forecasting, 36*(1), 54–74.

Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2021). The M5 accuracy competition: Results, findings, and conclusions. *International Journal of Forecasting.*

Makridakis, S., Wheelwright, S. C., & Hyndman, R. J. (1998). *Forecasting: Methods and applications* (3rd ed.). Wiley.

Matthews, B. W. (1975). Comparison of the predicted and observed secondary structure of T4 phage lysozyme. *Biochimica et Biophysica Acta (BBA) - Protein Structure, 405*(2), 442–451.

Nelson, D. M., Pereira, A. C., & de Oliveira, R. A. (2017). Stock market's price movement prediction with LSTM neural networks. In *2017 International Joint Conference on Neural Networks (IJCNN)* (pp. 1419–1426).

Pandey, A., Mannepalli, P. K., Gupta, M., & Others. (2024). A deep learning based hybrid CNN LSTM model for location aware web service recommendation. *Neural Processing Letters, 56*, 234. https://doi.org/10.1007/s11063-024-11687-w

Powers, D. M. W. (2011). Evaluation: From precision, recall and F-measure to ROC, informedness, markedness and correlation. *Journal of Machine Learning Technologies, 2*(1), 37–63.

Prechelt, L. (1998). Early stopping—But when? In G. Orr & K. Müller (Eds.), *Neural networks: Tricks of the trade* (pp. 55–69). Springer.

Reddy, D., & Karthik, A. (2021). *Forecasting stock price using LSTM-CNN method*. International Journal of Engineering and Advanced Technology, 11(1), 1–7. https://doi.org/10.35940/ijeat.A3117.1011121

Shi, X., Chen, Z., Wang, H., Yeung, D. Y., Wong, W. K., & Woo, W. C. (2015). Convolutional LSTM network: A machine learning approach for precipitation nowcasting. *Advances in Neural Information Processing Systems, 28.*

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management, 45*(4), 427–437.

Tashman, L. J. (2000). Out-of-sample tests of forecasting accuracy: An analysis and review. *International Journal of Forecasting, 16*(4), 437–450.

Tsay, R. S. (2010). *Analysis of financial time series* (3rd ed.). Wiley.

Zhang, G. P. (2003). Time series forecasting using a hybrid ARIMA and neural network model. *Neurocomputing, 50*, 159–175.

Code Availability

All code used in this dissertation, including data preprocessing, feature engineering, model architectures (ARIMA, LSTM, CNN-LSTM), training scripts, and walk-forward validation implementation, has been uploaded to a public GitHub repository:

https://github.com/EngMohamedKemany/stock-regime-prediction-deepmodels